

RRAM-PoolFormer: A Resistive Memristor-based PoolFormer Modeling and Training Framework for Edge-AI Applications

Tiancheng Cao^{1,2}, Weihao Yu³, Yuan Gao², Chen Liu², Shuicheng Yan⁴, Wang Ling Goh¹

¹School of Electrical and Electronic Engineering, Nanyang Technological University, Republic of Singapore

²Institute of Microelectronics (IME), Agency for Science, Technology and Research (A*STAR), Republic of Singapore

³ Department of Electrical and Computer Engineering, National University of Singapore, Republic of Singapore

⁴Sea AI Lab, Republic of Singapore

Email: CAOT0006@e.ntu.edu.sg

Abstract—PoolFormer is a type of neural network architecture that is abstracted from Transformer where the computationally heavy token mixer module is replaced with simple pooling function. This paper presents a memristor-based PoolFormer modeling and training framework for edge-AI applications. To fit for implementation on resistive crossbar array, original PoolFormer structure is further optimized by replacing normalization operation with hardware friendly scaling operation. In addition, the non-idealities of RRAM crossbar from device to array level as well as peripheral readout circuits are also included. By incorporating these elements under a single framework for network training, their impact to the network performance can be effectively mitigated. This framework is implemented in a combination of Python and PyTorch. A 16-block PoolFormer network is designed and optimized for CIFAR-10 image classification tasks using measured 64×64 RRAM crossbar array results. The total network weights are only 0.26M, which is at least one order of magnitude smaller than that of the conventional DNN implementation. When compared to the ideal model with FP64 weight bit-length, 85.86% inference accuracy is reached with only 4-level weight resolutions and less than 4% accuracy loss.

Keywords—Memristor crossbar, Transformer, PoolFormer, Edge-AI, neuromorphic system

I. INTRODUCTION

Transformer has demonstrated remarkable classification performance improvement in computer vision [1-6] and natural language processing [7-8]. The superior performance of Transformer is dependent on a lot of computing power required by the self-attention operation. Although customized Transformer accelerator chip [9-11] and cloud cluster [12] have been reported in literature, these solutions primarily aim for large-scale neural network applications. They are not suitable for lightweight neural network running on battery powered edge-AI devices which have limited storage memory, computation power and battery capacity. Recent studies unveiled that the competence of transformer model is largely attributed to its general architecture, rather than the self-attention mechanism itself. In [13], a new model called

PoolFormer is proposed by replacing the computationally heavy self-attention operation with simple pooling operation. Significant operation simplification is achieved with mild compromise in performance. Since it requires no parameters in the memory and the processing procedure is much simplified, tremendous amount of computation resources can be saved.

On the other hand, resistive memristor crossbar arrays have gained popularity recently because of their attractive features of high data throughput and excellent energy efficiency. Memristive devices have the unique ability to store weights due to their non-volatile programmable conductance. Additionally, by applying input voltage pulses and measuring the resulting output current, the vector-matrix multiplication (VMM) process can be performed in parallel, which results in a considerable acceleration of overall network computing [14-16]. Therefore, resistive crossbar array-based processing-in-memory (PIM) architecture is an elegant solution to enhance both computation speed and energy efficiency by several orders compared to conventional memory [17-18].

The implementations of Transformer structure on memristor crossbar array have been reported in literature [23]. There are also efforts to build hardware-software co-design framework with full circuit models to simulate and evaluate the performance of Transformer network in circuit implementation. For example, the self-attention operation is decomposed to reduce the frequency of intermediate results reloading and transfers the Softmax function to logical inference with lookup table [24]. However, these reported solutions still require considerable computation resources.

Inspired by the success of PoolFormer in computer vision, we aim to exploit its application to edge-AI and the hardware implementation on memristive crossbar array in this work. A modeling and training framework called RRAM-PoolFormer is proposed. The key innovations in this work can be summarized as follows. (1) Proposed a hardware friendly PoolFormer structure suitable for implementation on memristor crossbar. Specifically, self-attention operation is replaced with pooling as an extremely weak token mixer. (2) Non-ideality of memristor from device to array and readout circuits are incorporated into network training process to effectively compensate PoolFormer

This work was supported by the Agency for Science, Technology and research (A*STAR), Singapore under the Nanosystems at the Edge programme, grant no. A18A1b0055.

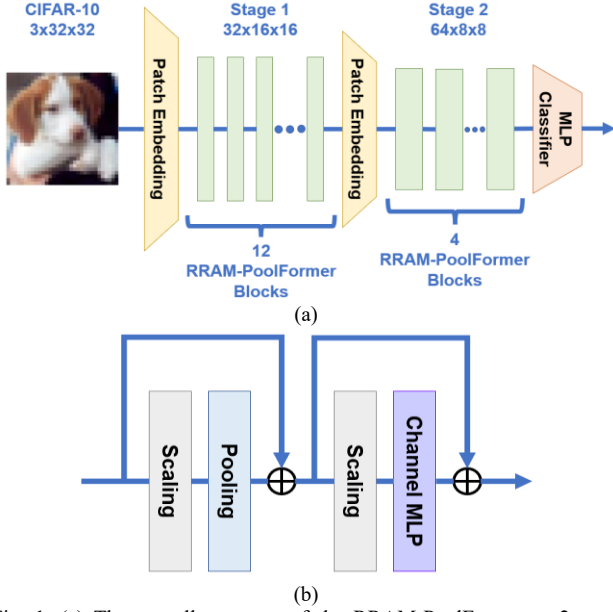


Fig. 1. (a) The overall structure of the RRAM-PoolFormer, a 2-stage structure with 12 and 4 blocks. (b) Architecture of RRAM-PoolFormer block, consisting of scaling, pooling and channel MLP operations.

design and optimization. To verify the efficacy of the proposed framework, a 16-block RRAM-PoolFormer network is designed for CIFAR-10 image classification task [25] with low resolution 4-level 64×64 RRAM arrays, the network size is at least one order smaller and yet the inference results is comparable to the conventional CNN based memristor accelerator models [26-28].

The rest of the paper is organized as follows. Section II introduce the structure of proposed RRAM-PoolFormer and its implementation on crossbar array. Section III presents the non-idealities aware training framework. Section IV shows the simulation results and Section V concludes the paper.

II. RRAM-POOLFORMER DESIGN AND IMPLEMENTATION

A. RRAM-PoolFormer Structure

The structure of the overall RRAM-PoolFormer network is shown in Fig. 1(a). The RRAM block architecture is given in Fig. 1(b). Compared to the conventional Transformer, the proposed structure has following novelties to adapt for edge device applications:

(1) To balance the performance of network and the circuit efficiency, layer normalization operation is replaced by channel scaling. The scaling factor for each channel is optimized automatically during the network training process and then applied to the inference tasks. Compared to the conventional normalization operation, channel scaling can maintain satisfactory accuracy on small-size datasets for edge-AI applications while the computation requirement can be much relaxed.

(2) Standard PoolFormer involves 4 stages, each stage has $L/6$, $L/6$, $L/2$ and $L/6$ blocks to process the data, where L is the total number of blocks. Since edge device applications focus

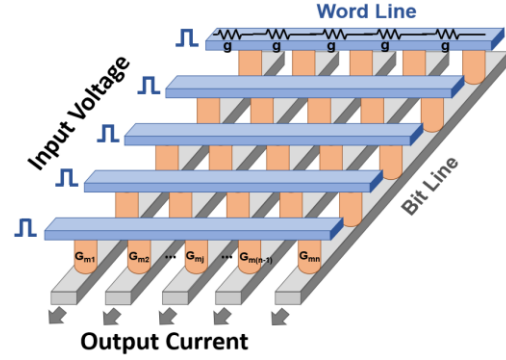


Fig. 2. A block diagram of a $m \times n$ crossbar array with interconnection line resistance is comprised of horizontal word lines and vertical bit lines. Line resistance represents the resistance in ohms of the interconnect paths that run through the top and bottom electrodes of the memristor devices.

more on small datasets, requiring less storage and computation resources. Hence, 2-stage structure with 12 and 4 blocks is selected in the proposed RRAM-PoolFormer. Moreover, the embedding dimension is chosen to be 32 and 64 for each stage to reduce the total number of parameters. The final optimized model has 2 stages with 16×16 and 8×8 tokens.

B. Crossbar Array structure

A typical memristor crossbar array is shown in Fig. 2. It comprises of memristor devices at each cross point along with a group of lines that consist of horizontal word lines (WL) and vertical bit lines (BL). Line resistance describes the resistance in ohms of the connection pathways that run through the top and bottom parts of memristor devices. Using a deep submicron CMOS technology, the unit line conductance g between two neighboring memristor devices is in the region of a few Ohms [29].

When applying an input voltage vector V_{IN} to WL, if the effect of line resistance is disregarded, the output current of j^{th} column I_{Oj} can be expressed as follows

$$I_{Oj} = \sum_{i=1}^m V_{INi} \cdot G_{ij} \quad (1)$$

where $1 \leq i \leq m, 1 \leq j \leq n$. V_{INi} is the input voltage of i^{th} row, G_{ij} is the memristor device conductance at i^{th} row, j^{th} column.

For crossbar-based computing, the mapping of negative weights is an unavoidable problem. Several earlier works have provided the one-cell-one-weight method with reference current [30]. Due to the fluctuation of devices, it is currently not feasible for large-scale arrays. The two-cell-one-weight method depicted is thus used [16]. One weight is used demonstrated by every two neighboring cells. Cells in the right column denote negative weight, whereas cells in the left column represent positive weight. To determine the real output current I_O , the difference of bit line current of two columns is sensed.

$$I_O = I_O^+ - I_O^- = V_{IN} \times (G^+ - G^-) \quad (2)$$

C. Operation Implementation

1) Patch Embedding

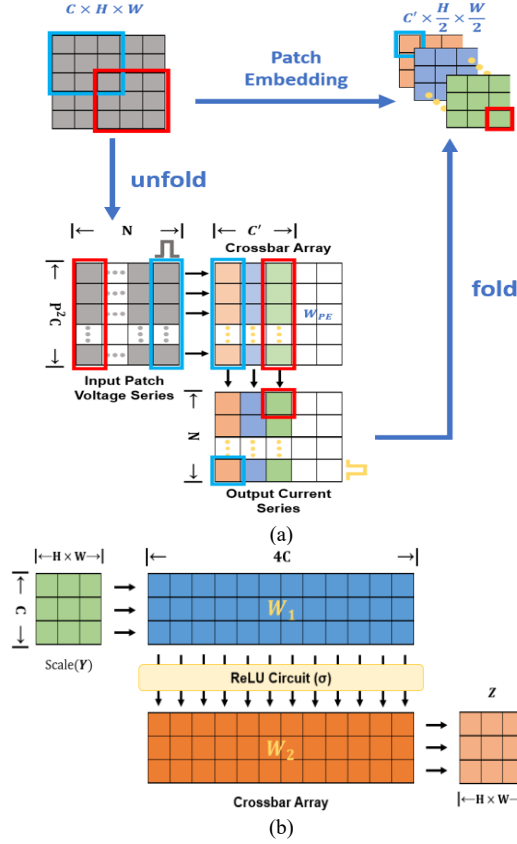


Fig. 3. Implementation of (a) Patch Embedding and (b) Channel MLP.

Patch Embedding (PE) [3] is used to downscale the sample and extract the features before each stage.

$$X = PE(I) = fold(unfold(I) \times W_{PE}) \quad (3)$$

where $I \in (H, W, C)$ and $X \in (\frac{H}{2}, \frac{W}{2}, C')$, W_{PE} is the learnable weight, P is the path size, C is the input channel, N is the path number and C' is the embedding dimension. Patch size is fixed at 3 and the stride is fixed at 2 in this work. As shown in Fig. 3(a), to implement the embedding process, the input is firstly unfolded to N patches with size (N, P^2C) . Then, each patch is converted to the input voltage of the crossbar array and fed into the word lines. After collecting the VMM results with size (N, C') , the output of embedding is folded to feature maps with size $(\frac{H}{2}, \frac{W}{2}, C')$.

2) Channel MLP

The channel MLP block consists of 2-layer MLP with a scaling and non-linear active function. The detail formula is

$$Z = \sigma(Scale(Y) \times W_1) \times W_2 \quad (4)$$

where $W_1 \in R^{C \times 4C}$ and $W_2 \in R^{4C \times C}$ are learnable parameters in MLP. $\sigma(\cdot)$ denotes the non-linear active function and ReLU is used in this work.

The weights of MLP layer are mapped on crossbar array directly in Fig. 3(b). When the input voltage is applied to word lines, the result of vector matrix multiplication is collected as

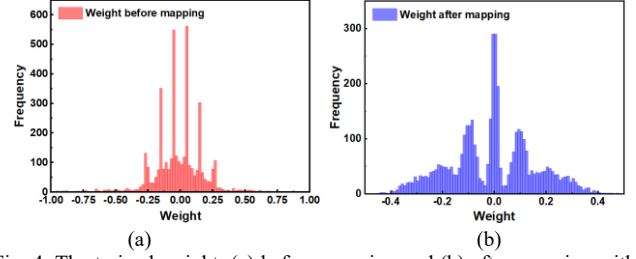


Fig. 4. The trained weight: (a) before mapping and (b) after mapping with nonidealities.

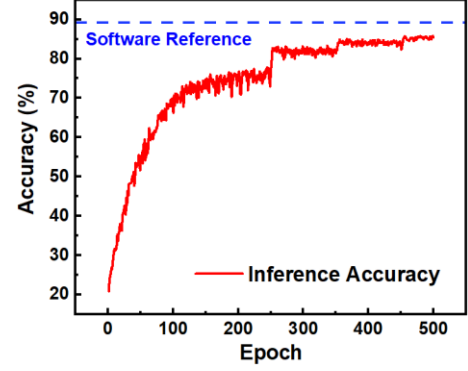


Fig. 5. In the inference system, the effects of all non-idealities are assessed concurrently. Red line shows the inference accuracy during the training and blue line refers to the software accuracy as reference.

output current from the bit lines. At the same time, the ReLU circuit in recent work [31] helps to transfer the bit line current to the input voltage of the second layer MLP.

III. IMPACT OF CROSSBAR ARRAY NONIDEALITIES

A. Device Non-idealities

Due to manufacturing imperfections, non-ideal memristor device characteristics such conductance variation, nonlinearity and programming failure are inevitable, and they can bring significant impact to the network performance [32]. Therefore, the non-idealities should be considered during the network design stage and compensated appropriately. In each training cycle, a random mask matrix that represents programming failure is generated based on the recorded programming failure rate, to simulate its impact. The array conductance matrix and this programming failure mask are combined using AND operation to get the programmed conductance value. It should be noted that HRS status is assigned to the failed devices.

B. Crossbar Non-idealities

Sneak path and line resistance are the major contributors to crossbar array non-idealities [33]. Because of the sneak pathways and cumulative IR-drop on the interconnect traces, there will be difference in node voltage between the theoretical and actual cases. Conductance overlapping and line resistance will combine, as seen in Fig. 4, to obscure the true conductance levels and reduce the accuracy.

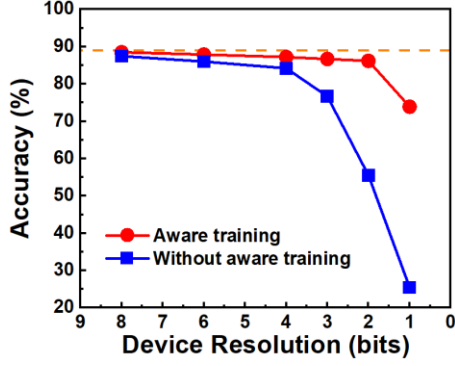


Fig. 6. The impact of the device resolution to inference accuracy.

Iterative and linear techniques have been used in several similar research to predict IR-drop in crossbar [34-35]. Applying the Kirchhoff's laws equations to calculate the crossbar current distribution is the usual solution. Iterative matrix algorithms can lower the time complexity to $O(m^2n^2)$. Even so, it still takes a lot of time, especially as the crossbar size grows up. A novel strategy was put forth to handle the voltage and current degradations as scaling factor matrices that are applied to the output current and input voltage in order to improve simulation speed. This model successfully combined quick computation with $O(mn)$ time complexity and low current distortion is obtained. Further technical information is explained in [36].

IV. SIMULATION RESULTS

The memristor model is built with the measured TaO_x 2T1R RRAM crossbar results [37]. 4-level device resistance in the range of 5 k Ω to 30 k Ω is used. After the weight mapping to crossbar array, the non-idealities-aware training is carried out [31]. It took 2.7 hours to be trained on a 16-core 3.2GHz AMD Ryzen 5800H CPU with 32GB RAM. For inference operation, the effects of all non-idealities are assessed holistically and the inference accuracy is shown in Fig. 5. With 64 $\times$ 64 array size, 0.5 Ω line resistance and 15% device variation, the inference accuracy reaches 85.86% within 500 epochs.

Fig. 6 illustrates how the device resolution affects training accuracy. After mapping the weights to 2-8 bit resolutions, the proposed training method with 2-bit weight only loses 3% of inference accuracy compared to ideal software result. However, conventional offline training has much higher accuracy degradation. Fig. 7 displays the effect of line resistance on inference accuracy. As the line resistance increases from 0 Ω to 2 Ω , the accuracy of the conventional training approach experiences a substantial drop due to the change in voltage distribution brought about by the line resistance. On the other hand, the non-ideality aware training method experiences a slight decrease in accuracy of less than 1%, still retaining similar accuracy.

The results of this study are summarized and compared to other recent memristor crossbar DNN solutions in Table I. For an equitable assessment of network performance, the results for the CIFAR-10 image classification task among the reported

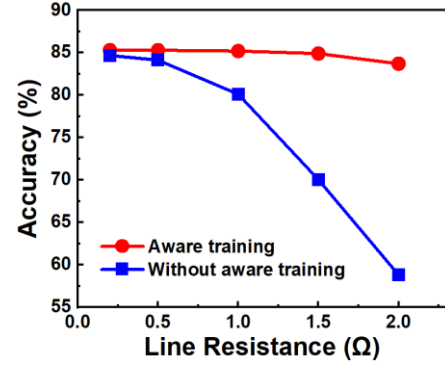


Fig. 7. The impact of the crossbar array line resistance to inference accuracy with 2-bit resolution.

Table. I. Performance Comparison

	[25]	[26]	[27]		This Work
Network	CNN	CNN	CNN	CNN	Transformer
Model	VGG-11	VGG	VGG-8	VGG-8	PoolFormer
Device	HfO ₂	HfO ₂ /Al ₂ O ₃	EpiRAM	TaO _x /HfO _x	TaO _x
Memristor Levels	NR	16	64	128	4
Ron (Ω)	11k	50k	81k	100k	5k
On/Off Ratio	1.9	16	50.2	10	6
Variation (σ/μ)	NR	4%	2%	3.7%	15%
Number of Weights	7.66M	6.6M	13M	13M	0.26M
Line Res. (Ω)	NR	0.1	NR	NR	0.5
Crossbar Size	256 $\times$ 256	1152 $\times$ 128	128 $\times$ 128	128 $\times$ 128	64 $\times$ 64
Inference Accuracy (%)	85.2	85.17	NR	NR	85.86

NR: Not reported

works are compared. It was found that the proposed framework achieved outstanding inference accuracy of 85.86% using only 4-level memristors, and the overall network size was significantly reduced to only 0.26M weights, which is at least one order smaller compared to the other state-of-art designs.

V. CONCLUSION

In this study, a PoolFormer modeling and training framework for edge AI applications using memristors has been presented. Furthermore, a framework that takes into account non-idealities for software-hardware co-design of edge devices based on memristive crossbars is described. The 16-layer RRAM-PoolFormer with nonidealities achieved 85.86% inference accuracy on the CIFAR-10 dataset while consuming fewer resources compared to conventional deep neural network implementations. This proposed technique offers a comprehensive circuit solution to assess the performance of the RRAM-PoolFormer model and determine the requirement for low-resolution memristor crossbars for edge AI vision applications.

REFERENCES

- [1] A. Vaswani, et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NIPS)*, 2017.
- [2] N. Carion and F. Massa, et al., "End-to-end object detection with Transformers," in *European conference on computer vision (ECCV)*, 2020, pp. 213-229.
- [3] A. Dosovitskiy and L. Beyer, et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2020.
- [4] Z. Liu and Y. Lin, et al., "Swin Transformer: Hierarchical vision Transformer using shifted windows," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 10012-10022.
- [5] L. Yuan and Y. Chen, et al., "Tokens-to-token vit: Training vision Transformers from scratch on imagenet," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 558-567.
- [6] P. Ramachandran and N. Parmar, et al., "Stand-alone self-attention in vision models," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 32, 2019.
- [7] J. Devlin and M. W. Chang, et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171-4186.
- [8] T. Brown and B. Mann, et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 33, pp. 1877-1901, 2020.
- [9] Y. -H. Chen, T. Krishna, J. S. Emer and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127-138, Jan. 2017.
- [10] N.P. Jouppi, et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Archit. (ISCA)*, pp. 1-12, Jun. 2017.
- [11] J. Lee, et al., "UNPU: an energy-efficient deep neural network accelerator with fully variable weight bit precision," *IEEE J. Solid-State Circuits*, vol. 54, no. 1, pp. 173-185, Jan. 2019.
- [12] J. Fowers et al., "A configurable cloud-scale DNN processor for Real-Time AI," *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pp. 1-14, 2018.
- [13] W. Yu, M. Luo, P. Zhou, C. Si, Y. Zhou, X. Wang, J. Feng, and S. Yan, "Metaformer is actually what you need for vision," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10819-10829.
- [14] X. Wang, M. A. Zidan and W. D. Lu, "A crossbar-based in-memory computing architecture," *IEEE Trans. on Circuits and Systems I: Regular Papers*, vol. 67, no. 12, pp. 4224-4232, Dec. 2020.
- [15] W. Zhang, B. Gao, J. Tang et al., "Neuro-inspired computing chips," *Nat Electron* 3, 371-382, 2020.
- [16] S. Yu, "Neuro-inspired computing with emerging nonvolatile memories," *Proceedings of the IEEE*, vol. 106, no. 2, pp. 260-285, Feb. 2018.
- [17] Kim, H., et al. "4K-memristor analog-grade passive crossbar circuit," *Nature Comm.*, vol. 12, no. 1, p. 5198, Aug. 2021.
- [18] F. Kiani, et al. "A fully hardware-based memristive multilayer neural network," *Science Advances*, vol. 7, no. 48, p. eabj4801, 2021.
- [19] H.-S. P. Wong et al., "Phase change memory," *Proc. IEEE*, vol. 98, no. 12, pp. 2201-2227, Dec. 2010.
- [20] D. Apalkov, B. Dieny and J. M. Slaughter, "Magnetoresistive random access memory," *Proc. IEEE*, vol. 104, no. 10, pp. 1796-1830, Oct. 2016.
- [21] H. -S. P. Wong et al., "Metal-oxide RRAM," *Proc. IEEE*, vol. 100, no. 6, pp. 1951-1970, June 2012.
- [22] T. Cao, W. L. Goh and Y. Gao, "Performance analysis of convolutional neural network using multi-level memristor crossbar for edge computing," *2020 3rd International Conference on Intelligent Autonomous Systems (ICoIAS)*, pp. 107-111, 2020.
- [23] M. Kang, H. Shin and L. -S. Kim, "A framework for accelerating Transformer-based language model on ReRAM-based architecture," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 41, no. 9, pp. 3026-3039, Sept. 2022.
- [24] X. Yang, B. Yan, H. Li and Y. Chen, "ReTransformer: ReRAM-based processing-in-memory architecture for Transformer acceleration," *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1-9.
- [25] Alex Krizhevsky (2009). Learning Multiple Layers of Features from Tiny Images. [Online]. Available: <https://www.cs.toronto.edu/~kriz/cifar.html>.
- [26] L. Xia, M. Liu, X. Ning, K. Chakrabarty and Y. Wang, "Fault-tolerant training enabled by on-line fault detection for RRAM-based neural computing systems," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 38, no. 9, pp. 1611-1624, Sept. 2019.
- [27] X. Fei, Y. Zhang and W. Zheng, "XB-SIM*: A simulation framework for modeling and exploration of ReRAM-based CNN acceleration design," *Tsinghua Science and Technology*, vol. 26, no. 3, pp. 322-334, June 2021.
- [28] X. Peng, S. Huang, H. Jiang, A. Lu and S. Yu, "DNN+NeuroSim V2.0: An end-to-end benchmarking framework for compute-in-memory accelerators for on-chip training," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* vol. 40, no. 11, pp. 2306-2319, Nov. 2021.
- [29] ITRS, "International technology roadmap for semiconductors," 2013.
- [30] W. Wang et al., "Efficient training of the memristive deep belief net immune to non-idealities of the synaptic devices," *Advanced Intelligent Systems*, Feb. 2022.
- [31] T. Cao, C. Liu, Y. Gao and W. L. Goh, "A non-idealities aware software-hardware co-design framework for edge-AI deep neural network implemented on memristive crossbar," *IEEE J. Emerging Sel. Top. Circuits Syst.*, Oct 2022.
- [32] X. Sun and S. Yu, "Impact of non-ideal characteristics of resistive synaptic devices on implementing convolutional neural networks," *IEEE J. Emerging Sel. Top. Circuits Syst.*, vol. 9, no. 3, pp. 570-579, Sept. 2019.
- [33] T. Cao, C. Liu, Y. Gao and W. L. Goh, "Parasitic-aware modelling for neural networks implemented with memristor crossbar array," *2021 IEEE 14th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*, pp. 122-126, 2021.
- [34] B. Liu et al., "Reduction and IR-drop compensations techniques for reliable neuromorphic computing systems," *2014 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2014.
- [35] M. E. Fouda, J. Lee, A. M. Eltawil and F. Kurdahi, "Overcoming crossbar nonidealities in binary neural networks through learning," *2018 IEEE/ACM International Symposium on Nanoscale Architectures (NANOARCH)*, Athens, Greece, 2018.
- [36] T. Cao, C. Liu, Y. Gao and W. L. Goh, "Parasitic-aware modeling and neural network training scheme for energy-efficient processing-in-memory with resistive crossbar array," *IEEE J. Emerging Sel. Top. Circuits Syst.*, vol. 12, no. 2, pp. 436-444, Jun. 2022.
- [37] W. Wang, et al., "Enabling neuromorphic computing: BEOL integration of CMOS RRAM chip and programmable performance," *2019 IEEE International System-on-Chip Conference (SOCC)*, Singapore, pp. 354-358, 2019.